

Using the `cryst tract` Package for Crystallographic Analysis

Don Ngo, Julia Maria Hubner, Anirudh Prabhu

2025-11-19

Contents

Introduction	1
1. The Core <code>cryst tract</code> Workflow	2
2. Tools for Post-Processing and Analysis	10
3. Advanced: High-Throughput Batch Processing	15
Conclusion	19

Introduction

This vignette demonstrates the core workflow for using the `cryst tract` package. The package is designed for the **batch processing** of Crystallographic Information Files (.cif) to extract structural and chemical information from the repeating atomic arrangement that defines a crystal. It calculates key geometric properties like bond lengths and angles and compiles the results into a structured format suitable for large-scale data analysis.

We will first demonstrate the main batch-processing function, `analyze_cif_files`. Then, to illustrate how it works, we will walk through the analysis of a single file step-by-step, explaining the underlying functions, the crystallographic principles, and the mathematical formulas they employ.

A Note on Example Data

Due to licensing restrictions, the example CIF files from the ICSD are no longer bundled with the `cryst tract` package. To follow the examples in this vignette, you must download the required data yourself.

1. **Find Required Files:** A complete list of necessary files can be found in the `required_ICSD_files.csv` file at the root of the `ml-crystals` repository. For this introductory vignette, you will need **ICSD #422**.
2. **Download Data:** Access the ICSD database (requires a license) and download the .cif file for entry 422.
3. **Store the File:** Save the file as `ICSD422.cif` in a known location on your computer.
4. **Update Paths:** In the code examples below, you will need to replace the placeholder path `"path/to/your/ICSD422.cif"` with the actual path to your downloaded file.

To run the code chunks yourself, you must also remove the `eval=TRUE` option from the chunk headers.

Setup: Loading the Package

First, we load the `cryst tract` package.

```
library(crystract)
```

1. The Core `cryst tract` Workflow

The primary goal of `cryst tract` is to automate the analysis of many CIF files at once. While the package provides granular functions for each step of the crystallographic analysis, the most common use case is to leverage the main wrapper function for an end-to-end pipeline.

1.1 The Full Pipeline for Batch Processing

The `analyze_cif_files()` function is the cornerstone of the package. It performs the entire sequence of operations—from reading files to calculating bond angles with error propagation—for one or more files. It takes a vector of file paths and returns a single `data.table` where each row corresponds to a processed CIF file. Complex results (like coordinates, distances, and angles) are stored in list-columns for easy access and analysis.

This is the recommended function for batch processing.

```
# IMPORTANT: Update this path to point to your downloaded CIF file.
cif_path <- "~/repos/ml-crystals/packages/crystract/inst/extdata/ICSD422.cif"

# Run the pipeline on our single example file
analysis_results <- analyze_cif_files(cif_path)
#> Starting analysis of 1 files in 1 batches.
#>
#> --- Processing Batch 1 of 1 (Files 1 to 1) ---
#> -----
#> Analysis Complete!
#> Successfully processed a total of 1 structures.

# Let's inspect the structure of the output table.
# It's a single row containing all our results in nested data.tables.
str(analysis_results, max.level = 2)
#> Classes 'data.table' and 'data.frame':  1 obs. of  17 variables:
#>  $ file_name                : chr "ICSD422.cif"
#>  $ database_code            : chr "ICSD 422"
#>  $ chemical_formula         : chr "Si1 Sr2"
#>  $ structure_type           : chr "TiNiSi#MgSrSi"
#>  $ space_group_name         : chr "P n m a"
#>  $ space_group_number       : chr "62"
#>  $ unit_cell_metrics        :List of 1
#> ..$ :Classes 'data.table' and 'data.frame':  1 obs. of  12 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#>  $ atomic_coordinates       :List of 1
#> ..$ :Classes 'data.table' and 'data.frame':  3 obs. of  11 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#>  $ symmetry_operations      :List of 1
#> ..$ :Classes 'data.table' and 'data.frame':  8 obs. of  3 variables:
```

```

#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> $ transformed_coords :List of 1
#> ..$ :Classes 'data.table' and 'data.frame': 12 obs. of 4 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> $ expanded_coords :List of 1
#> ..$ :Classes 'data.table' and 'data.frame': 324 obs. of 4 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> $ distances :List of 1
#> ..$ :Classes 'data.table' and 'data.frame': 969 obs. of 6 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> $ bonded_pairs_minimum_distance:List of 1
#> ..$ :Classes 'data.table' and 'data.frame': 14 obs. of 9 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> .. ..- attr(*, "sorted")= chr [1:3] "Atom1" "Atom2" "Distance"
#> $ bonded_pairs_brunner :List of 1
#> ..$ : NULL
#> $ bonded_pairs_hoppe :List of 1
#> ..$ : NULL
#> $ neighbor_counts :List of 1
#> ..$ :Classes 'data.table' and 'data.frame': 3 obs. of 2 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> $ bond_angles :List of 1
#> ..$ :Classes 'data.table' and 'data.frame': 30 obs. of 5 variables:
#> .. ..- attr(*, ".internal.selfref")=<externalptr>
#> .. ..- attr(*, "sorted")= chr [1:3] "CentralAtom" "Neighbor1" "Neighbor2"
#> - attr(*, ".internal.selfref")=<externalptr>

```

As shown in the structure output, the result is a tidy **data.table** containing all extracted and calculated information. We can easily access the nested data frames for further analysis. Note that the output includes separate results for each bonding algorithm requested, such as **bonded_pairs_minimum_distance**, **bonded_pairs_brunner**, and **bonded_pairs_hoppe**.

To get the final bonded pairs table from the Minimum Distance method (which includes propagated errors):

```

# The result is a list-column, so we access the element with [[1]]
final_bonds <- analysis_results$bonded_pairs_minimum_distance[[1]]

print(head(final_bonds))
#> Key: <Atom1, Atom2, Distance>
#>   Atom1      Atom2 Distance DeltaX DeltaY DeltaZ      dcut      dmin
#>   <char>      <fctr>    <num>  <num>  <num>  <num>    <num>    <num>
#> 1:   Si1 Sr1_4_0_-1_-1 3.184477 -0.0932   0.5  0.1797 3.479899 3.163544
#> 2:   Si1 Sr1_4_0_0_-1 3.184477 -0.0932  -0.5  0.1797 3.479899 3.163544
#> 3:   Si1 Sr2_1_0_0_-1 3.261366  0.2347   0.0  0.2776 3.479899 3.163544
#> 4:   Si1 Sr2_3_-1_-1_0 3.465249  0.2731   0.5 -0.0720 3.479899 3.163544
#> 5:   Si1 Sr2_3_-1_0_0 3.465249  0.2731  -0.5 -0.0720 3.479899 3.163544
#> 6:   Si1 Sr1_1_0_0_0 3.163544  0.1010   0.0 -0.3203 3.479899 3.163544
#>   DistanceError
#>   <num>
#> 1:  0.008313329
#> 2:  0.008313329
#> 3:  0.014072755
#> 4:  0.009293087

```

```
#> 5: 0.009293087
#> 6: 0.014160750
```

1.2 A Step-by-Step Walkthrough

To understand what `analyze_cif_files()` does under the hood, this section breaks down the process. We will use a single CIF file to demonstrate each function individually, explaining the crystallographic concepts and the structure of the output at each stage.

1.2.1 Loading CIF Data We use the package's `read_cif_files()` function to load data into memory. For this demonstration, we use an example CIF file for Strontium Silicide (Sr_2Si) from the Inorganic Crystal Structure Database (ICSD), specifically entry 422. This entry can be found online here: [ICSD 422](#).

```
# The path was defined in the previous section:
cif_data_list <- read_cif_files(cif_path)

# We'll work with the content of the first file
cif_content <- cif_data_list[[1]]

# Let's look at the first few lines of the raw data
knitr::kable(
  head(cif_content),
  caption = "First 6 lines of the raw CIF data."
)
```

Table 1: First 6 lines of the raw CIF data.

```
V1
#(C) 2023 by FIZ Karlsruhe - Leibniz Institute for Information Infrastructure. All rights reserved.
data_422-ICSD
_database_code_ICSD 422
_audit_creation_date 1980-01-01
_audit_update_record 2000-07-15
_chemical_name_common 'Strontium silicide (2/1)'
```

1.2.2 Extracting Metadata and Unit Cell Parameters A crystal structure is defined by its **unit cell** and the arrangement of atoms within it. We first extract this high-level information.

```
database_code <- extract_database_code(cif_content)
chemical_formula <- extract_chemical_formula(cif_content)
space_group_name <- extract_space_group_name(cif_content)
space_group_number <- extract_space_group_number(cif_content)

cat("Database Code:", database_code, "\n")
#> Database Code: ICSD 422
cat("Chemical Formula:", chemical_formula, "\n")
#> Chemical Formula: Si1 Sr2
cat("Space Group:", space_group_name, "(No.", space_group_number, ")\n")
#> Space Group: P n m a (No. 62 )
```

Next, `extract_unit_cell_metrics()` extracts the six parameters that define the shape and size of the unit cell: the lengths of its three axes (a , b , c) and the angles between them (α , β , γ). Their experimental uncertainties are also extracted.

```
unit_cell_metrics <- extract_unit_cell_metrics(cif_content)
print(unit_cell_metrics)
#>   _cell_length_a _cell_length_b _cell_length_c _cell_angle_alpha
#>           <num>           <num>           <num>           <num>
#> 1:           8.11           5.15           9.54           90
#>   _cell_angle_beta _cell_angle_gamma _cell_length_a_error
#>           <num>           <num>           <num>
#> 1:           90           90           NA
#>   _cell_length_b_error _cell_length_c_error _cell_angle_alpha_error
#>           <num>           <num>           <num>
#> 1:           NA           NA           NA
#>   _cell_angle_beta_error _cell_angle_gamma_error
#>           <num>           <num>
#> 1:           NA           NA
```

1.2.3 Extracting Atomic and Symmetry Data Instead of listing every atom in the unit cell, a CIF file efficiently describes the structure using only the **asymmetric unit**: the smallest set of unique atoms. All other atoms in the unit cell can be generated by applying the crystal's **symmetry operations** to this unique set.

```
# Extract the coordinates of the unique atoms in the asymmetric unit
atomic_coordinates <- extract_atomic_coordinates(cif_content)
print("Asymmetric Atomic Coordinates:")
#> [1] "Asymmetric Atomic Coordinates:"
print(atomic_coordinates)
#>   Label WyckoffSymbol WyckoffMultiplicity Occupancy OccupancyError
#>   <char>      <char>           <num>      <num>           <num>
#> 1:  Sr1          c              4          1             NA
#> 2:  Sr2          c              4          1             NA
#> 3:  Si1          c              4          1             NA
#>   x_a   y_b   z_c x_error y_error z_error
#>   <num> <num> <num> <num>  <num>  <num>
#> 1: 0.6529 0.25 0.0769 0.0006    NA 0.0005
#> 2: 0.5192 0.25 0.6748 0.0006    NA 0.0005
#> 3: 0.2539 0.25 0.1028 0.0016    NA 0.0014

# Extract the symmetry operations
symmetry_operations <- extract_symmetry_operations(cif_content)
print("Symmetry Operations (first 6 of 8):")
#> [1] "Symmetry Operations (first 6 of 8):"
print(head(symmetry_operations))
#>   x      y      z
#>   <char> <char> <char>
#> 1: x+1/2      y -z+1/2
#> 2:      x -y+1/2      z
#> 3: -x+1/2  y+1/2  z+1/2
#> 4:      -x      -y      -z
#> 5: -x+1/2      -y  z+1/2
#> 6:      -x  y+1/2      -z
```

1.2.4 Generating the Full Crystal Structure Now we use the asymmetric atoms and symmetry operations to computationally build the full crystal structure. This is a two-step process.

Formula Context: Symmetry Operations

A symmetry operation is an affine transformation that maps an initial fractional coordinate (x, y, z) to a new coordinate (x', y', z') . It consists of a rotation/reflection matrix $\mathbf{W}$ and a translation vector $\mathbf{w}$.

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} \quad \text{or} \quad \mathbf{x}' = \mathbf{W}\mathbf{x} + \mathbf{w}$$

The `apply_symmetry_operations` function applies all of the crystal's symmetry operations to each atom in the asymmetric unit, generating the complete set of atoms within the primary unit cell.

Formula Context: Supercell Expansion

To find all nearest neighbors of an atom, we must also consider atoms in adjacent unit cells. The `expand_transformed_coords` function generates a **supercell** (in this case, 3x3x3) by translating the primary unit cell atoms by integer vectors (i, j, k) , where i, j, k each range from -1 to 1. An atom at fractional coordinate (x, y, z) generates new coordinates:

$$(x_{exp}, y_{exp}, z_{exp}) = (x + i, y + j, z + k)$$

```
# Apply symmetry to generate all atoms in the primary unit cell
transformed_coords <- apply_symmetry_operations(atomic_coordinates, symmetry_operations)
print("Unique atoms in full unit cell (first 6 of 12):")
#> [1] "Unique atoms in full unit cell (first 6 of 12):"
print(head(transformed_coords))
#>      Label    x_a    y_b    z_c
#>      <char> <num> <num> <num>
#> 1: Sr1_1 0.1529 0.25 0.4231
#> 2: Sr1_2 0.6529 0.25 0.0769
#> 3: Sr1_3 0.8471 0.75 0.5769
#> 4: Sr1_4 0.3471 0.75 0.9231
#> 5: Sr2_1 0.0192 0.25 0.8252
#> 6: Sr2_2 0.5192 0.25 0.6748

# Expand into a 3x3x3 supercell for neighbor calculations
expanded_coords <- expand_transformed_coords(transformed_coords)
print("Atoms in supercell (first 6 of 324):")
#> [1] "Atoms in supercell (first 6 of 324):"
print(head(expanded_coords))
#>      Label    x_a    y_b    z_c
#>      <char> <num> <num> <num>
#> 1: Sr1_1_-1_-1_-1 -0.8471 -0.75 -0.5769
#> 2: Sr1_2_-1_-1_-1 -0.3471 -0.75 -0.9231
#> 3: Sr1_3_-1_-1_-1 -0.1529 -0.25 -0.4231
#> 4: Sr1_4_-1_-1_-1 -0.6529 -0.25 -0.0769
#> 5: Sr2_1_-1_-1_-1 -0.9808 -0.75 -0.1748
#> 6: Sr2_2_-1_-1_-1 -0.4808 -0.75 -0.3252
```

1.2.5 Calculating Interatomic Distances Formula Context: Interatomic Distance in a Triclinic System

Because unit cell axes are not always orthogonal, the simple Pythagorean theorem is insufficient. The distance d between two atoms at fractional coordinates (x_{f1}, y_{f1}, z_{f1}) and (x_{f2}, y_{f2}, z_{f2}) requires the full crystallographic distance formula:

$$d = \left(\begin{aligned} &a^2(x_{f1} - x_{f2})^2 + b^2(y_{f1} - y_{f2})^2 + c^2(z_{f1} - z_{f2})^2 \\ &+ 2ab(x_{f1} - x_{f2})(y_{f1} - y_{f2})\cos\gamma \\ &+ 2ac(x_{f1} - x_{f2})(z_{f1} - z_{f2})\cos\beta \\ &+ 2bc(y_{f1} - y_{f2})(z_{f1} - z_{f2})\cos\alpha \end{aligned} \right)^{1/2}$$

The `calculate_distances` function computes the distances from each central atom (from the asymmetric unit) to all other atoms in the generated supercell.

```
distances <- calculate_distances(atomic_coordinates, expanded_coords, unit_cell_metrics)
print("Calculated Distances (shortest 6):")
#> [1] "Calculated Distances (shortest 6):"
print(head(distances[order(Distance)]))
#>      Atom1      Atom2 Distance DeltaX DeltaY DeltaZ
#>      <fctr>      <fctr>      <num>  <num>  <num>  <num>
#> 1:   Si1   Sr1_1_0_0_0 3.163544  0.1010   0.0 -0.3203
#> 2:   Sr1   Si1_1_0_0_0 3.163544 -0.1010   0.0 -0.3203
#> 3:   Sr1 Si1_4_0_-1_-1 3.184477 -0.0932   0.5  0.1797
#> 4:   Sr1 Si1_4_0_-1_-1 3.184477 -0.0932  -0.5  0.1797
#> 5:   Si1 Sr1_4_0_-1_-1 3.184477 -0.0932   0.5  0.1797
#> 6:   Si1 Sr1_4_0_-1_-1 3.184477 -0.0932  -0.5  0.1797
```

1.2.6 Identifying Bonds and Neighbors **Formula Context: Bonding and Coordination Number**

Identifying which atoms are “bonded” is key to determining the **coordination number (CN)**. `crystract` implements several methods:

- **minimum_distance (Default)**: Defines a custom distance cutoff for each central atom i :

$$d_i^{\text{cut}} = (1 + \delta)d_i^{\text{min}}$$

Here, d_i^{min} is the shortest distance from atom i to any other atom, and δ is a tolerance parameter (default is 0.1).

- **brunner**: Identifies bonds by finding the largest gap in the reciprocal distances.
- **hoppe (ECoN)**: Uses an iterative method to calculate Effective Coordination Numbers, weighting neighbor contributions by distance.

The `minimum_distance` function filters the `distances` table to identify bonded pairs. `calculate_neighbor_counts` then summarizes these to find the integer CN for each central atom.

```
# Identify bonded pairs using the minimum distance method with a tolerance of 10%
bonded_pairs <- minimum_distance(distances, delta = 0.1)
print("Bonded Pairs (first 6):")
#> [1] "Bonded Pairs (first 6):"
print(head(bonded_pairs))
#>      Atom1      Atom2 Distance DeltaX DeltaY DeltaZ      dcut      dmin
#>      <fctr>      <fctr>      <num>  <num>  <num>  <num>      <num>      <num>
#> 1:   Sr1 Si1_4_0_-1_-1 3.184477 -0.0932   0.5  0.1797 3.479899 3.163544
#> 2:   Sr1 Si1_4_0_-1_-1 3.184477 -0.0932  -0.5  0.1797 3.479899 3.163544
```

```
#> 3:   Sr1   Si1_1_0_0_0 3.163544 -0.1010    0.0 -0.3203 3.479899 3.163544
#> 4:   Sr1   Si1_2_0_0_0 3.245310  0.3990    0.0 -0.0259 3.479899 3.163544
#> 5:   Sr2  Si1_3_0_-1_0 3.465249  0.2731    0.5  0.0720 3.587503 3.261366
#> 6:   Sr2  Si1_1_0_0_0 3.261366 -0.2347    0.0  0.2776 3.587503 3.261366

# Calculate neighbor counts based on the bonded pairs
neighbor_counts <- calculate_neighbor_counts(bonded_pairs)
print("Neighbor Counts:")
#> [1] "Neighbor Counts:"
print(neighbor_counts)
#>      Atom NeighborCount
#>   <fctr>      <int>
#> 1:   Sr1              4
#> 2:   Sr2              3
#> 3:   Si1              7
```

1.2.7 Calculating Bond Angles Formula Context: Bond Angle Calculation

To calculate a bond angle A-B-C (with B at the vertex), the fractional coordinates must first be converted to an orthogonal Cartesian system.

1. **Fractional to Cartesian Conversion:** A fractional coordinate ($\mathbf{x}_f$) is converted to a Cartesian coordinate ($\mathbf{x}_c$) via a transformation matrix $\mathbf{M}$:

$$\begin{pmatrix} x_c \\ y_c \\ z_c \end{pmatrix} = \mathbf{M} \cdot \begin{pmatrix} x_f \\ y_f \\ z_f \end{pmatrix} \quad \text{where} \quad \mathbf{M} = \begin{pmatrix} a & b \cos \gamma & \frac{c \cos \beta}{\sin \gamma} \\ 0 & b \sin \gamma & \frac{c(\cos \alpha - \cos \beta \cos \gamma)}{ab \sin \gamma} \\ 0 & 0 & \frac{V}{ab \sin \gamma} \end{pmatrix}$$

where V is the unit cell volume.

2. **Angle via Dot Product:** With atoms in Cartesian space, the angle θ between vectors $\vec{u}$ (from B to A) and $\vec{v}$ (from B to C) is:

$$\theta = \arccos \left(\frac{\vec{u} \cdot \vec{v}}{|\vec{u}| |\vec{v}|} \right)$$

The `calculate_angles` function implements this for all possible bond angles around each central atom.

```
bond_angles <- calculate_angles(
  bonded_pairs,
  atomic_coordinates,
  expanded_coords,
  unit_cell_metrics
)
print("Calculated Bond Angles (first 6):")
#> [1] "Calculated Bond Angles (first 6):"
print(head(bond_angles))
#>   CentralAtom Neighbor1 Neighbor2 Angle
#>   <char>      <fctr>      <fctr>      <num>
#> 1:      Si1 Sr1_4_0_-1_-1 Sr1_4_0_0_-1 107.92071
#> 2:      Si1 Sr1_4_0_-1_-1 Sr2_1_0_0_-1  72.62529
#> 3:      Si1 Sr1_4_0_-1_-1 Sr2_3_-1_-1_0  69.97350
#> 4:      Si1 Sr1_4_0_-1_-1 Sr2_3_-1_0_0 149.23688
#> 5:      Si1 Sr1_4_0_-1_-1 Sr1_1_0_0_0 125.55190
#> 6:      Si1 Sr1_4_0_-1_-1 Sr1_2_0_0_0  73.87978
```

1.2.8 Error Propagation Finally, `crystract` propagates the experimental uncertainties from cell parameters and atomic coordinates to the calculated distances and angles.

Formula Context: Error Propagation

The uncertainty, σ_f , in a calculated value f that depends on several uncorrelated variables p_i (each with uncertainty σ_{p_i}) is found using the sum of squares of the partial derivatives:

$$\sigma_f^2 = \sum_i \left(\frac{\partial f}{\partial p_i} \sigma_{p_i} \right)^2$$

Uncertainty in Interatomic Distance (σ_d) The uncertainty in the distance, σ_d , is found by applying the general formula to the distance d . The input parameters p_i are the 12 variables that define the distance: $\{a, b, c, \alpha, \beta, \gamma, x_{f1}, y_{f1}, z_{f1}, x_{f2}, y_{f2}, z_{f2}\}$. Letting $\Delta x = x_{f1} - x_{f2}$, etc., the partial derivatives are, for example:

$$\frac{\partial d}{\partial a} = \frac{1}{2d} (2a(\Delta x)^2 + 2b(\Delta x)(\Delta y) \cos \gamma + 2c(\Delta x)(\Delta z) \cos \beta)$$

Uncertainty in Bond Angle (σ_θ) Propagating error to the bond angle θ is a more complex, multi-step process involving propagating initial errors first to Cartesian coordinates, and then from those Cartesian uncertainties to the final angle via the chain rule.

```
# Propagate errors for interatomic distances
bonded_pairs_with_error <- propagate_distance_error(
  bonded_pairs,
  atomic_coordinates,
  unit_cell_metrics
)
print("Bonded Pairs with Distance Error (first 6):")
#> [1] "Bonded Pairs with Distance Error (first 6):"
print(head(bonded_pairs_with_error))
#> Key: <Atom1, Atom2, Distance>
#>      Atom1      Atom2 Distance  DeltaX DeltaY DeltaZ      dcut      dmin
#>      <char>      <fctr>    <num>    <num>  <num>  <num>    <num>    <num>
#> 1:   Si1 Sr1_4_0_-1_-1 3.184477 -0.0932   0.5  0.1797 3.479899 3.163544
#> 2:   Si1 Sr1_4_0_0_-1 3.184477 -0.0932  -0.5  0.1797 3.479899 3.163544
#> 3:   Si1 Sr2_1_0_0_-1 3.261366  0.2347   0.0  0.2776 3.479899 3.163544
#> 4:   Si1 Sr2_3_-1_-1_0 3.465249  0.2731   0.5 -0.0720 3.479899 3.163544
#> 5:   Si1 Sr2_3_-1_0_0 3.465249  0.2731  -0.5 -0.0720 3.479899 3.163544
#> 6:   Si1 Sr1_1_0_0_0 3.163544  0.1010   0.0 -0.3203 3.479899 3.163544
#>      DistanceError
#>      <num>
#> 1:  0.008313329
#> 2:  0.008313329
#> 3:  0.014072755
#> 4:  0.009293087
#> 5:  0.009293087
#> 6:  0.014160750

# Propagate errors for bond angles
bond_angles_with_error <- propagate_angle_error(
  bond_angles,
  atomic_coordinates,
  expanded_coords,
```

```

unit_cell_metrics
)
print("Bond Angles with Angle Error (first 6):")
#> [1] "Bond Angles with Angle Error (first 6):"
print(head(bond_angles_with_error))
#> Key: <CentralAtom, Neighbor1, Neighbor2>
#>   CentralAtom Neighbor1 Neighbor2 Angle AngleError
#>   <char>      <fctr>      <fctr>      <num>      <num>
#> 1:      Si1 Sr1_4_0_-1_-1 Sr1_4_0_0_-1 107.92071 0.3991814
#> 2:      Si1 Sr1_4_0_-1_-1 Sr2_1_0_0_-1 72.62529 0.2625336
#> 3:      Si1 Sr1_4_0_-1_-1 Sr2_3_-1_-1_0 69.97350 0.1320523
#> 4:      Si1 Sr1_4_0_-1_-1 Sr2_3_-1_0_0 149.23688 0.4492550
#> 5:      Si1 Sr1_4_0_-1_-1 Sr1_1_0_0_0 125.55190 0.2121372
#> 6:      Si1 Sr1_4_0_-1_-1 Sr1_2_0_0_0 73.87978 0.2610475

```

2. Tools for Post-Processing and Analysis

After running the main analysis pipeline, **cryststruct** provides several helper functions to filter and refine the results, allowing you to isolate data based on chemical identity or key crystallographic properties.

2.1 Filtering by Chemical Identity

The `filter_atoms_by_symbol()` function provides an interactive way to filter results (like bond or angle tables) to focus on the coordination environment around a specific type of element.

```

# In an interactive R session, you would run this:
filtered_bonds <- filter_atoms_by_symbol(
  data_table = bonded_pairs_with_error,
  atom_col = "Atom1" # Filter based on the central atom
)
#> Available base atom symbols found: Si, Sr
#> Please enter the chemical symbols you want to filter for, separated by commas (e.g., C,Si,O):
#> No symbols entered. Returning an empty data table.

```

If you were to type **Si** and press Enter, the function would return a new **data.table** containing only the rows where the central atom (**Atom1**) is Silicon.

2.2 Filtering by Wyckoff Position

For many crystallographic studies, it is crucial to analyze atoms based on their specific site symmetry, described by their **Wyckoff position** (e.g., “2a”, “6c”). The `filter_by_wyckoff_symbol()` function is designed for this purpose.

```

# 1. In our example, all asymmetric atoms occupy the Wyckoff site 'c' with multiplicity 4 ("4c").
print("Atomic coordinates showing Wyckoff information:")
#> [1] "Atomic coordinates showing Wyckoff information:"
print(atomic_coordinates[, .(Label, WyckoffSymbol, WyckoffMultiplicity)])
#>   Label WyckoffSymbol WyckoffMultiplicity
#>   <char>      <char>      <num>
#> 1:   Sr1           c           4
#> 2:   Sr2           c           4

```

```
#> 3:      Si1          c          4
cat("\n")

# 2. Filter bonds where the central atom is on the "4c" Wyckoff site.
bonds_from_4c_site <- filter_by_wyckoff_symbol(
  data_table = bonded_pairs_with_error,
  atomic_coordinates = atomic_coordinates,
  atom_col = "Atom1",
  wyckoff_symbols = "4c"
)

print(paste("Number of rows in original bond table:", nrow(bonded_pairs_with_error)))
#> [1] "Number of rows in original bond table: 14"
print(paste("Number of rows after filtering for site '4c':", nrow(bonds_from_4c_site)))
#> [1] "Number of rows after filtering for site '4c': 14"
```

2.3 Filtering Ghost Distances Using Atomic Radii

A common issue in crystallography is **site-occupancy disorder**, where a single crystallographic site is statistically co-occupied by different atoms. Standard calculations can produce physically meaningless, short “ghost” distances.

The `filter_ghost_distances()` function cleans the distance table by removing these artifacts. It uses a built-in table of covalent radii to establish a plausible bond length range. Any calculated distance falling outside this physical range (defined by a `margin`) is filtered out.

```
# A distance d is kept if: (r1+r2)*(1-margin) <= d <= (r1+r2)*(1+margin)
filtered_result <- filter_ghost_distances(
  distances = distances,
  atomic_coordinates = atomic_coordinates,
  margin = 0.1 # Default margin is 10%
)

kept_distances <- filtered_result$kept
removed_distances <- filtered_result$removed

cat("Total distances calculated:", nrow(distances), "\n")
#> Total distances calculated: 969
cat("Distances kept after filtering:", nrow(kept_distances), "\n")
#> Distances kept after filtering: 24
cat("Ghost distances removed:", nrow(removed_distances), "\n\n")
#> Ghost distances removed: 945

# For a well-ordered structure like Sr2Si, the 'removed' table should be empty.
print("Removed ghost distances (should be empty for this well-ordered example):")
#> [1] "Removed ghost distances (should be empty for this well-ordered example):"
print(removed_distances)
#>      Atom1      Atom2 Distance expected_dist lower_bound upper_bound
#>      <fctr>      <fctr>      <num>      <num>      <num>      <num>
#> 1:   Si1 Si1_1_-1_-1_-1  9.395616      2.34      2.106      2.574
#> 2:   Si1 Si1_2_-1_-1_-1 13.539062      2.34      2.106      2.574
#> 3:   Si1 Si1_3_-1_-1_-1  9.807429      2.34      2.106      2.574
#> 4:   Si1 Si1_4_-1_-1_-1  5.238116      2.34      2.106      2.574
```

```

#> 5: Si1 Si1_1_0_-1_-1 9.395616 2.34 2.106 2.574
#> ---
#> 941: Sr2 Sr2_4_0_1_1 9.913241 3.84 3.456 4.224
#> 942: Sr2 Sr2_1_1_1_1 12.783275 3.84 3.456 4.224
#> 943: Sr2 Sr2_2_1_1_1 13.539062 3.84 3.456 4.224
#> 944: Sr2 Sr2_3_1_1_1 14.931034 3.84 3.456 4.224
#> 945: Sr2 Sr2_4_1_1_1 12.609249 3.84 3.456 4.224
#> Reason
#> <char>
#> 1: Distance is TOO LONG
#> 2: Distance is TOO LONG
#> 3: Distance is TOO LONG
#> 4: Distance is TOO LONG
#> 5: Distance is TOO LONG
#> ---
#> 941: Distance is TOO LONG
#> 942: Distance is TOO LONG
#> 943: Distance is TOO LONG
#> 944: Distance is TOO LONG
#> 945: Distance is TOO LONG

```

2.4 Filtering by Element Exclusion

To focus an analysis on a specific part of a structure (e.g., a host framework without guest atoms), `filter_by_elements()` allows for the complete removal of bonds involving certain elements.

```

# Let's filter our bond table to exclude any bonds involving Strontium ("Sr").
# Since all bonds in this structure are Si-Sr, the result should be an empty table.
bonds_without_sr <- filter_by_elements(
  distances = bonded_pairs_with_error,
  atomic_coordinates = atomic_coordinates,
  elements_to_exclude = "Sr"
)

cat("Number of bonds in original table:", nrow(bonded_pairs_with_error), "\n")
#> Number of bonds in original table: 14
cat("Number of bonds after excluding 'Sr':", nrow(bonds_without_sr), "\n")
#> Number of bonds after excluding 'Sr': 0

```

2.5 Calculating Weighted Average Network Bond Distance

A common goal is to compute a single summary statistic that represents a structure.

The `calculate_weighted_average_network_distance()` function calculates a representative **bond length** for a specified atomic network (defined by a set of Wyckoff sites).

This calculation should always be performed on a table of identified **bonds** to ensure the result is physically meaningful.

Formula Context: Weighted Average Network Distance To obtain a single, representative bond length for an atomic network, the function calculates a true weighted average over all individual bonds in the unit cell. This is equivalent to calculating the **total sum of all bond lengths** within the network

and dividing by the **total number of bonds**. This method correctly accounts for site multiplicity, site occupancy, and varying coordination numbers.

The formula is defined as:

$$\bar{d}_{\text{network}} = \frac{\sum_j (m_j \cdot \text{occ}_j \cdot S_j)}{\sum_j (m_j \cdot \text{occ}_j \cdot n_j)}$$

Where, for each unique atomic site j in the asymmetric unit: - m_j is the **Wyckoff multiplicity** (how many equivalent atoms are generated by symmetry). - occ_j is the **site occupancy** (accounts for disorder; 1.0 for a fully occupied site). - n_j is the **coordination number** (the number of bonds for an atom at site j). - S_j is the **sum of all bond distances** for a single atom at site j . It is calculated as:

$$S_j = \sum_{i=1}^{n_j} d_{ij}$$

The numerator, $\sum_j (m_j \cdot \text{occ}_j \cdot S_j)$, represents the grand total sum of the lengths of every bond in the unit cell. The denominator, $\sum_j (m_j \cdot \text{occ}_j \cdot n_j)$, represents the total number of bonds in the unit cell.

Best Practices for Accurate Calculation For the most physically meaningful and accurate results, it is crucial to perform this calculation as the final step in a filtering pipeline, especially when dealing with complex or disordered structures. The recommended workflow is:

1. Calculate all potential interatomic distances using `calculate_distances()`.
2. Clean this raw data by removing non-physical “ghost” distances with `filter_ghost_distances()`.
3. Identify the true bonded pairs from the cleaned data using an algorithm like `minimum_distance()`.
4. **Only then**, pass the resulting table of confirmed bonds to `calculate_weighted_average_network_distance()`.

This “Ghosts -> Bonds -> Average” sequence ensures the final metric is calculated only over actual chemical bonds, preventing artifacts from skewing the result.

```
# Calculate the weighted average BOND distance for the entire Sr2Si network.
# First, identify the bonds in the structure. We use the `bonded_pairs` table
# created in section 1.2.6.

# Then, define the Wyckoff sites belonging to the network. Here, it's just "4c".
# Note: The function expects the full Wyckoff symbol including multiplicity.
network_wyckoff_sites <- "4c"

# Apply the function to the table of identified bonds.
# For this simple, ordered structure, all occupancies are 1.0, but the function
# correctly applies the full formula.
weighted_avg_bond_dist <- calculate_weighted_average_network_distance(
  distances = bonded_pairs, # Use the bond table as input
  atomic_coordinates = atomic_coordinates,
  wyckoff_symbols = network_wyckoff_sites
)

cat("Weighted average network bond distance for the '4c' sites:", weighted_avg_bond_dist, "Å\n")
#> Weighted average network bond distance for the '4c' sites: 3.281382 Å
```

2.6 Exporting Results to CSV Files

After performing a batch analysis, you are left with a `data.table` where many columns are themselves nested `data.tables`. While this is convenient for interactive analysis in R, it is not ideal for sharing or for use with other software.

The `export_analysis_to_csv()` function solves this by unpacking the entire results table into a structured directory of CSV files. It creates a main directory and, within it, sub-directories for each type of data (e.g., `atomic_coordinates`, `bond_angles`, `distances`). Each sub-directory will contain one CSV file per analyzed CIF, named after the original file. A `meta` folder is also created to store a summary CSV of the non-nested metadata.

This makes the results easily accessible and portable.

```
# 1. We use the 'analysis_results' table from the main workflow.

# 2. Define a temporary output directory for this example.
export_path <- file.path(tempdir(), "crystract_export")

# 3. Export the results. 'overwrite = TRUE' is used to allow re-running the example.
export_analysis_to_csv(analysis_results, export_path, overwrite = TRUE)
#> Analysis successfully exported to: /private/var/folders/b9/kfunx8ps2jv7msmszzg0r_1r0000gp/T/RtmpplN2

# 4. We can list the created files and folders to see the structure.
cat("Exported directory structure:\n")
#> Exported directory structure:
print(list.files(export_path, recursive = TRUE))
#> [1] "atomic_coordinates/ICSD422.csv"
#> [2] "bond_angles/ICSD422.csv"
#> [3] "bonded_pairs_minimum_distance/ICSD422.csv"
#> [4] "distances/ICSD422.csv"
#> [5] "expanded_coords/ICSD422.csv"
#> [6] "meta/meta_summary.csv"
#> [7] "neighbor_counts/ICSD422.csv"
#> [8] "symmetry_operations/ICSD422.csv"
#> [9] "transformed_coords/ICSD422.csv"
#> [10] "unit_cell_metrics/ICSD422.csv"

# 5. Clean up the temporary directory after the example.
unlink(export_path, recursive = TRUE)
```

2.7 Customizing Atomic Radii

The `crystract` package includes a comprehensive table of covalent radii for use with `filter_ghost_distances()`. However, you may want to use a different set of radii, such as ionic radii, or add a missing element. The `set_radii_data()` function provides a flexible way to replace the default data for your entire R session.

A custom radii table must contain three columns: `Symbol`, `Radius`, and `Type`. This allows you to create a single table with multiple radius types and select the one you need at analysis time.

Let's demonstrate by creating a small table with both covalent and ionic radii for Sodium (Na) and Chlorine (Cl).

```
# 1. Create a custom radii data.table
my_custom_radii <- data.table::data.table(
```

```

Symbol = c("Na", "Na", "Cl", "Cl"),
Radius = c(1.54, 1.02, 0.99, 1.81),
Type = c("covalent", "ionic", "covalent", "ionic")
)

# 2. Set this table as the active table for this R session
set_radai_data(my_custom_radai)
#> Custom radai table has been set for the current R session.

# 3. Now, if you were to call filter_ghost_distances(), you could specify
# which radius type to use. For example, to filter based on ionic radai:
#
# filter_ghost_distances(distances, atomic_coordinates, radai_type = "ionic")
#
# This would use radai of 1.02 Å for Na and 1.81 Å for Cl.
# If you omit the argument, it defaults to "covalent" (1.54 Å and 0.99 Å).

# 4. To reset to the package's default table at any time:
set_radai_data(NULL)
#> Session radai data reset to package default.

```

This feature provides full control over the criteria used for identifying and filtering physically implausible bond distances, making the analysis adaptable to a wide range of chemical systems.

3. Advanced: High-Throughput Batch Processing

While the previous examples demonstrated `cryststruct` on a single file to explain the underlying mechanics, the package is optimized for processing thousands of structures simultaneously.

For large-scale datasets (like the AMCSO or ICSD), `cryststruct` utilizes the `future` package to parallelize operations across multiple CPU cores. It also supports a “Batch-to-Disk” mode, which saves results incrementally to avoid memory exhaustion during massive jobs.

3.1 Running a Parallel Batch Job

To process a directory containing many CIF files, you simply point `analyze_cif_files` to the directory or a list of file paths, specify the number of workers (cores), and provide an output directory.

```

# 1. Define your data source
# Assume we have a folder named 'cif_data' containing many .cif files
cif_folder <- "~/repos/ml-crystals/notebook/workflow/data"
all_cif_paths <- list.files(path = cif_folder, pattern = "\\..cif$", full.names = TRUE)

# 2. Run the analysis in parallel
# We use 'output_dir' to save batches to disk automatically.
# We use 'workers' to utilize 4 CPU cores.
analyze_cif_files(
  file_paths = all_cif_paths,
  workers = 4,                                # Number of parallel cores
  output_dir = "batch_results",               # Directory where results will be saved
  batch_size = 10,                           # Process in chunks of 10 files

```

```

# Pass arguments to control the analysis details
bonding_algorithms = "minimum_distance"
)
#> Setting parallel plan to use 4 workers.
#> Starting analysis of 150 files in 15 batches.
#>
#> --- Processing Batch 1 of 15 (Files 1 to 10) ---
#> Warning in value[[3L]](cond): A critical error occurred while processing
#> '1004007.cif': Join results in 250 rows; more than 189 = nrow(x)+nrow(i).
#> Check for duplicate key values in i each of which join to the same group
#> in x over and over again. If that's ok, try by=EACHI to run j for each
#> group to avoid the large allocation. If you are sure you wish to proceed,
#> rerun with allow.cartesian=TRUE. Otherwise, please search for this error
#> message in the FAQ, Wiki, Stack Overflow and data.table issue tracker for
#> advice.
#> Batch 1 complete. Saved 9 results to 'batch_results/batch_1.rds'.
#>
#> --- Processing Batch 2 of 15 (Files 11 to 20) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'OW' -> 'O'
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: H
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'NaM' -> 'Na', 'LiM' -> 'Li', 'CaM'
#> -> 'Ca', 'NaM' -> 'Na', 'MgM' -> 'Mg', 'FeM' -> 'Fe', 'AlM' -> 'Al', 'ZnM'
#> -> 'Zn', 'MnM' -> 'Mn', 'SiT' -> 'Si'
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'NaA' -> 'Na', 'NaM' -> 'Na'
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'CaX' -> 'Ca', 'FeY' -> 'Fe', 'MgY' ->
#> 'Mg', 'AlZ' -> 'Al', 'FeZ' -> 'Fe', 'BT' -> 'B', 'SiT' -> 'Si'
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'SiT' -> 'Si', 'AlT' -> 'Al', 'MgM' ->
#> 'Mg', 'MnM' -> 'Mn', 'TiM' -> 'Ti', 'ZnM' -> 'Zn', 'LiM' -> 'Li', 'NaM' ->
#> 'Na', 'KA' -> 'K', 'NaA' -> 'Na', 'KAm' -> 'K', 'NaAm' -> 'Na'
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: H
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'FeA' -> 'Fe', 'NbA' -> 'Nb', 'MnA' ->
#> 'Mn', 'TaA' -> 'Ta', 'NbB' -> 'Nb', 'FeB' -> 'Fe', 'TaB' -> 'Ta', 'MnB' ->
#> 'Mn'
#> Warning in value[[3L]](cond): A critical error occurred while processing
#> 'Ferrocolumbite_0003830.cif': Join results in 30 rows; more than 29 =
#> nrow(x)+nrow(i). Check for duplicate key values in i each of which join to
#> the same group in x over and over again. If that's ok, try by=EACHI to
#> run j for each group to avoid the large allocation. If you are sure you
#> wish to proceed, rerun with allow.cartesian=TRUE. Otherwise, please search
#> for this error message in the FAQ, Wiki, Stack Overflow and data.table
#> issue tracker for advice.
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'FeA' -> 'Fe', 'NbA' -> 'Nb', 'MnA' ->
#> 'Mn', 'TaA' -> 'Ta', 'NbB' -> 'Nb', 'FeB' -> 'Fe', 'TaB' -> 'Ta', 'MnB' ->
#> 'Mn'

```

```

#> Warning in value[[3L]](cond): A critical error occurred while processing
#> 'Ferrocolumbite_0003831.cif': Join results in 30 rows; more than 29 =
#> nrow(x)+nrow(i). Check for duplicate key values in i each of which join to
#> the same group in x over and over again. If that's ok, try by=.EACHI to
#> run j for each group to avoid the large allocation. If you are sure you
#> wish to proceed, rerun with allow.cartesian=TRUE. Otherwise, please search
#> for this error message in the FAQ, Wiki, Stack Overflow and data.table
#> issue tracker for advice.
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'FeA' -> 'Fe', 'MnA' -> 'Mn', 'NbA' ->
#> 'Nb', 'NbB' -> 'Nb', 'TaB' -> 'Ta', 'FeB' -> 'Fe', 'MnB' -> 'Mn'
#> Batch 2 complete. Saved 8 results to 'batch_results/batch_2.rds'.
#>
#> --- Processing Batch 3 of 15 (Files 21 to 30) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: H
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Corrected non-standard atom labels: 'CaX' -> 'Ca', 'NaX' -> 'Na', 'FeY' ->
#> 'Fe', 'MgY' -> 'Mg', 'TiY' -> 'Ti', 'AlZ' -> 'Al', 'MgZ' -> 'Mg', 'FeZ' ->
#> 'Fe', 'SiT' -> 'Si'
#> Batch 3 complete. Saved 10 results to 'batch_results/batch_3.rds'.
#>
#> --- Processing Batch 4 of 15 (Files 31 to 40) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Uncorrected non-standard atom labels remain: D
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Uncorrected non-standard atom labels remain: D
#> Batch 4 complete. Saved 10 results to 'batch_results/batch_4.rds'.
#>
#> --- Processing Batch 5 of 15 (Files 41 to 50) ---
#> Batch 5 complete. Saved 10 results to 'batch_results/batch_5.rds'.
#>
#> --- Processing Batch 6 of 15 (Files 51 to 60) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: Pb
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: Fe
#> Batch 6 complete. Saved 10 results to 'batch_results/batch_6.rds'.
#>
#> --- Processing Batch 7 of 15 (Files 61 to 70) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: H,
#> Li
#> Batch 7 complete. Saved 10 results to 'batch_results/batch_7.rds'.
#>
#> --- Processing Batch 8 of 15 (Files 71 to 80) ---
#> Batch 8 complete. Saved 10 results to 'batch_results/batch_8.rds'.
#>
#> --- Processing Batch 9 of 15 (Files 81 to 90) ---
#> Batch 9 complete. Saved 10 results to 'batch_results/batch_9.rds'.
#>
#> --- Processing Batch 10 of 15 (Files 91 to 100) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):

```

```

#> Implicit atoms likely present. Elements in formula not in coordinates: F
#> Batch 10 complete. Saved 10 results to 'batch_results/batch_10.rds'.
#>
#> --- Processing Batch 11 of 15 (Files 101 to 110) ---
#> Batch 11 complete. Saved 10 results to 'batch_results/batch_11.rds'.
#>
#> --- Processing Batch 12 of 15 (Files 111 to 120) ---
#> Batch 12 complete. Saved 10 results to 'batch_results/batch_12.rds'.
#>
#> --- Processing Batch 13 of 15 (Files 121 to 130) ---
#> Batch 13 complete. Saved 10 results to 'batch_results/batch_13.rds'.
#>
#> --- Processing Batch 14 of 15 (Files 131 to 140) ---
#> Warning in extract_atomic_coordinates(cif_content, chemical_formula):
#> Implicit atoms likely present. Elements in formula not in coordinates: H
#> Batch 14 complete. Saved 10 results to 'batch_results/batch_14.rds'.
#>
#> --- Processing Batch 15 of 15 (Files 141 to 150) ---
#> Warning in analyze_single_cif(cif_content = cif_content, file_name =
#> file_name, : Skipping calculations for file
#> 'JuH-WJ031-1a-8GPa-Na2B6Si2_optimierung-Na2B6Si2.cif' due to: missing
#> atomic coordinates.
#> Warning in value[[3L]](cond): A critical error occurred while processing
#> 'JuH-WJ031-1a-8GPa-Na2B6Si2_optimierung-Na2B6Si2.cif': object 'null' not
#> found
#> Batch 15 complete. Saved 9 results to 'batch_results/batch_15.rds'.
#> -----
#> Analysis Complete!
#> Batch results have been saved in the '/Users/donnngo/repos/ml-crystals/notebook/datathon/batch_results'

```

3.2 Aggregating Batch Results

Once the batch processing is complete, you will have a directory (`batch_results/`) filled with `.rds` files (e.g., `batch_1.rds`, `batch_2.rds`). `crystract` provides a helper function to merge these back into a single data table.

```

# Aggregate all batch files from the results directory
full_dataset <- aggregate_batch_results(
  input_dir = "batch_results",
  # Optional: Only keep specific columns to save memory
  # cols_to_keep = c("file_name", "chemical_formula", "bonded_pairs_minimum_distance", "bond_angles")
)
#> Found 15 batch files to aggregate.
#> Aggregation complete. Total rows: 146.

print(paste("Successfully loaded", nrow(full_dataset), "structures."))
#> [1] "Successfully loaded 146 structures."

```

This workflow ensures that even if a job involving an enormous amount structures is interrupted, previous batches are saved, and the parallelization drastically reduces computation time.

Conclusion

This vignette has demonstrated the core workflow of the `crystract` package, from the high-level, automated processing of multiple CIF files with `analyze_cif_files` to a detailed, step-by-step breakdown of the underlying calculations. We have seen how the package extracts fundamental data, builds a complete crystal model, calculates key geometric properties like bond distances and angles, and rigorously propagates experimental uncertainties into these final results.

Furthermore, with a powerful suite of post-processing tools such as `filter_atoms_by_symbol`, `filter_by_wyckoff_symbol`, `filter_ghost_distances`, `calculate_weighted_average_network_distance`, and `export_analysis_to_csv`, you can easily refine, clean, analyze, and share the generated datasets to focus on the specific chemical or crystallographic features of interest.

The goal of `crystract` is to provide a robust and accessible platform for crystallographic data mining in R.